José Ruiz Pareja **2/10/2026**

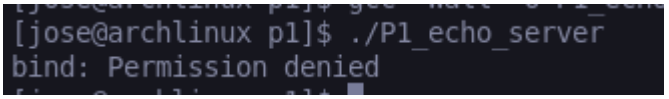
STAGE 1 Make it run

El primer error encontrado aparece al compilar el archivo, e incluso el propio IDE avisa del error:



Como bien nos sugiere al compilar, debemos cambiar “sin_prt” por “sin_port”.

El segundo error aparece al iniciar el ejecutable del servidor “P1_echo_server”.



Nos indica que no tenemos permiso, por lo que seguramente sea por usar un puerto privilegiado. Por lo que cambiamos el puerto que se utiliza (80) por un puerto mayor a 1024. Usaremos el puerto 9000 como aparece en el comentario de ejemplo del código del cliente.

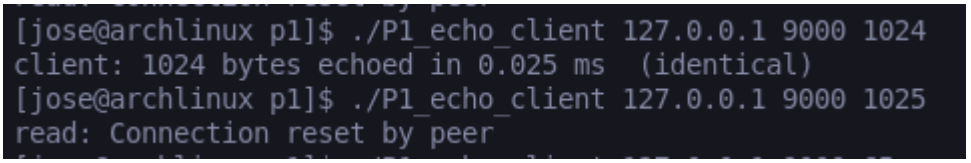
Línea cambiada en el código del servidor:

```
#define PORT 9000
```

Ejemplo del codigo del cliente:

```
*    example: ./P1_echo_client 127.0.0.1 9000 1024
```

El tercer error ocurre cuando enviamos un mensaje mayor a 1024 bytes, ya que el servidor solo realiza una vez el read, y en caso que queden bytes en el buffer los ignora.



Para arreglarlo he añadido un bucle justo después del read mientras n != 0, y añadiendo un read mas al final de él. En caso de que el buffer estuviese vacío NO realiza más iteraciones en el while, y si quedan bytes vuelve a entrar al bucle y los escribe.

```
n = read(conn_fd, buffer, BUFSIZE);
while (n != 0) {
  if (n < 0) {
    perror("read");
  } else {
    printf("server: received %zd bytes\n", n);
    fflush(stdout);
    if (write(conn_fd, buffer, n) < 0) perror("write");
    n = read(conn_fd, buffer, BUFSIZE);
  }
}
```

STAGE 2 Change one thing and measure

Tabla:

Size	Average time in ms	Chunks
10	0,0286	1
1000	0,0282	1
10000	0,0574	10
100000	40,640	98

El tiempo no crece en proporción al tamaño: pasar de 10 a 10000 bytes (1000 veces más datos) solo duplica el tiempo (de 0,029 a 0,057 ms), pero con 100000 bytes (10 veces más que 10000) cambia a unas 700 veces más porque el servidor lo atiende en 98 chunks en lugar de 10. Esa cadena de pequeñas lecturas y escrituras provoca las esperas.

STAGE 3 Break it on purpose

Primer experimento:

Parar el servidor mientras el cliente espera su respuesta. (Añadido sleep momentáneo para poder parar el servidor)

```
[jose@archlinux p1]$ ./P1_echo_server
server: listening on port 9000
server: client connected from 127.0.0.1:49172
server: received 1024 bytes
^C
```

El cliente:

```
[jose@archlinux p1]$ ./P1_echo_client 127.0.0.1 9000 10000
read: Connection reset by peer
```

El servidor cierra el socket y el cliente entra en un return y para también.

Segundo experimento:

Conectar el cliente a un puerto donde no hay un servidor escuchando.

```
[jose@archlinux p1]$ ./P1_echo_client 127.0.0.1 9001 10000
connect: Connection refused
```

El cliente no encuentra servidor al que conectarse y devuelve error

Tercer experimento:

Ejecutar dos clientes distintos a la vez al mismo servidor (añadido de nuevo un sleep(1) para poder ejecutar dos clientes a la vez)

Servidor:

```
[jose@archlinux p1]$ ./P1 echo server
[jose@archlinux p1]$ ./P1_echo_server
server: listening on port 9000
server: client connected from 127.0.0.1:53592
server: received 1024 bytes
server: received 1024 bytes
server: received 1024 bytes
server: received 928 bytes
server: client disconnected

server: client connected from 127.0.0.1:53602
server: received 1024 bytes
server: received 1024 bytes
server: received 1024 bytes
server: received 928 bytes
server: client disconnected
```

Cliente 1:

```
[jose@archlinux p1]$ ./P1_echo_client 127.0.0.1 9000 4000
client: 4000 bytes echoed in 3000.294 ms  (identical)
```

Cliente 2:

```
[jose@archlinux p1]$ ./P1_echo_client 127.0.0.1 9000 4000
client: 4000 bytes echoed in 6729.615 ms  (identical)
```

El servidor no puede manejar dos clientes a la vez, por lo que primero lee y escribe los bytes del mensaje enviado por el primer cliente en conectarse al servidor mientras el segundo cliente espera. Cuando termina el primer cliente, este cierra la conexión, y el segundo ya si realiza el intercambio con el servidor